Học sâu và ứng dụng
Mạng hồi quy
Recurrent neural network

Recurrent Neural Network là gì?

- Deep learning có 2 mô hình lớn
  - Convolutional Neural Network (CNN) cho bài toán có input là ảnh
  - Recurrent neural network (RNN) cho bài toán dữ liệu dạng chuỗi (sequence)

Recurrent Neural Network là gì?

- Bài toán: Cần phân loại hành động của người trong video, input là video 30s, output là phân loại hành động, ví dụ: đứng, ngồi, chạy, đánh nhau, bắn súng, ...
- Khi xử lý video ta hay gặp khái niệm FPS (frame per second) tức là bao nhiêu frame (ảnh) mỗi giây. Ví dụ 1 FPS với video 30s tức là lấy ra từ video 30 ảnh, mỗi giây một ảnh để xử lý.
- Ta dùng 1 FPS cho video input ở bài toán trên, tức là lấy ra 30 ảnh từ video, ảnh 1 ở giây 1, ảnh 2 ở giây 2,... ảnh 30 ở giây 30. Bây giờ input là 30 ảnh: ảnh 1, ảnh 2, ..., ảnh 30 và output là phân loại hành động.

Recurrent Neural Network là gì?

- Nhận xét:
  - Các ảnh có thứ tự: ảnh 1 xảy ra trước ảnh 2, ảnh 2 xảy ra trước ảnh 3, ...
    Nếu ta đảo lộn các ảnh thì có thể thay đổi nội dung của video.
  - Ta có thể dùng CNN để phân loại 1 ảnh trong 30 ảnh trên, nhưng rõ ràng là 1 ảnh không thể mô tả được nội dung của cả video.
    Ví dụ: Cảnh người cướp điện thoại, nếu ta chỉ dùng 1 ảnh là người đấy cầm điện thoại lúc cướp xong thì ta không thể biết được cả hành động cướp.

$\Rightarrow$ Cần một mô hình mới có thể giải quyết được bài toán với input là sequence (chuỗi ảnh 1->30)

$\Rightarrow$ Recurrent Neural Network (RNN) ra đời.

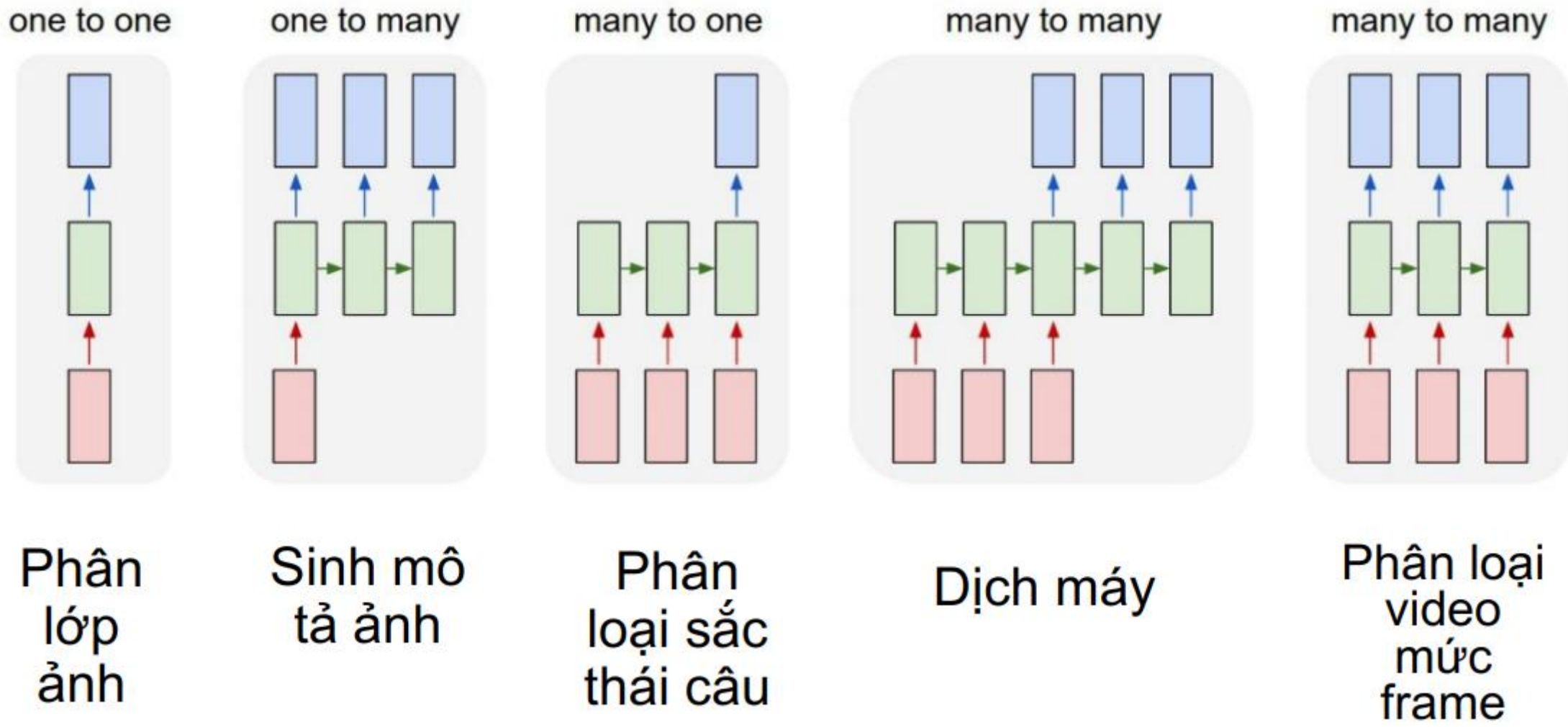
Recurrent Neural Network là gì?

- Dữ liệu dạng sequence:
  - Dữ liệu có thứ tự như các ảnh tách từ video ở trên được gọi là sequence, time-series data (xuất hiện đều đặn theo thứ tự thời gian).
  - Trong bài toán dự đoán đột quỵ tim cho bệnh nhân bằng các dữ liệu tim mạch khám trước đó. Input là dữ liệu của những lần khám trước đó, ví dụ i1 là lần khám tháng 1, i2 là lần khám tháng 2, ..., i8 là lần khám tháng 8. Các dữ liệu (i1, i2, …, i8) được gọi là sequence data hoặc time-series data. RNN sẽ học từ input và dự đoán xem bệnh nhân có bị đột quy tim hay không.
  - Ví dụ khác là trong bài toán dịch tự động với input là 1 câu, ví dụ “Tôi yêu Việt Nam" thì vị trí các từ và sự xắp xếp cực kì quan trọng đến nghĩa của câu và dữ liệu input [’Tôi’, ’yêu’, ’Việt’, ’Nam’] được gọi là sequence data.

    Trong bài toán xử lý ngôn ngữ (NLP) thì không thể xử lý cả câu được và người ta tách ra từng từ làm input, giống như trong video người ta tách ra các ảnh (frame) làm input.

Recurrent Neural Network là gì?

- Phân loại bài toán RNN:



Recurrent Neural Network là gì?

- Phân loại bài toán RNN:
  - **One to one**: mẫu bài toán cho Neural Network (NN) và Convolutional Neural Network (CNN), 1 input và 1 output, ví dụ với bài toán phân loại ảnh MNIST input là ảnh và output là ảnh đấy là số nào.

Recurrent Neural Network là gì?

- Phân loại bài toán RNN:
  - **One to many**: bài toán có 1 input nhưng nhiều output, ví dụ với bài toán caption cho ảnh, input là 1 ảnh nhưng output là nhiều chữ mô tả cho ảnh đó dưới dạng một câu.



"man in black shirt is playing guitar."



"construction worker in orange safety vest is working on road."



"two young girls are playing with lego toy."

Recurrent Neural Network là gì?

- Phân loại bài toán RNN:
  - **Many to one:** bài toán có nhiều input nhưng chỉ có 1 output, ví dụ bài toán phân loại hành động trong video, input là nhiều ảnh (frame) tách ra từ video, ouptut là hành động trong video.
  - **Many to many**: bài toán có nhiều input và nhiều output, ví dụ bài toán dịch từ tiếng Anh sang tiếng Việt, input là 1 câu gồm nhiều chữ: "I love Vietnam" và output cũng là 1 câu gồm nhiều chữ "Tôi yêu Việt Nam". Để ý là độ dài sequence của input và output có thể khác nhau.

Recurrent Neural Network là gì?

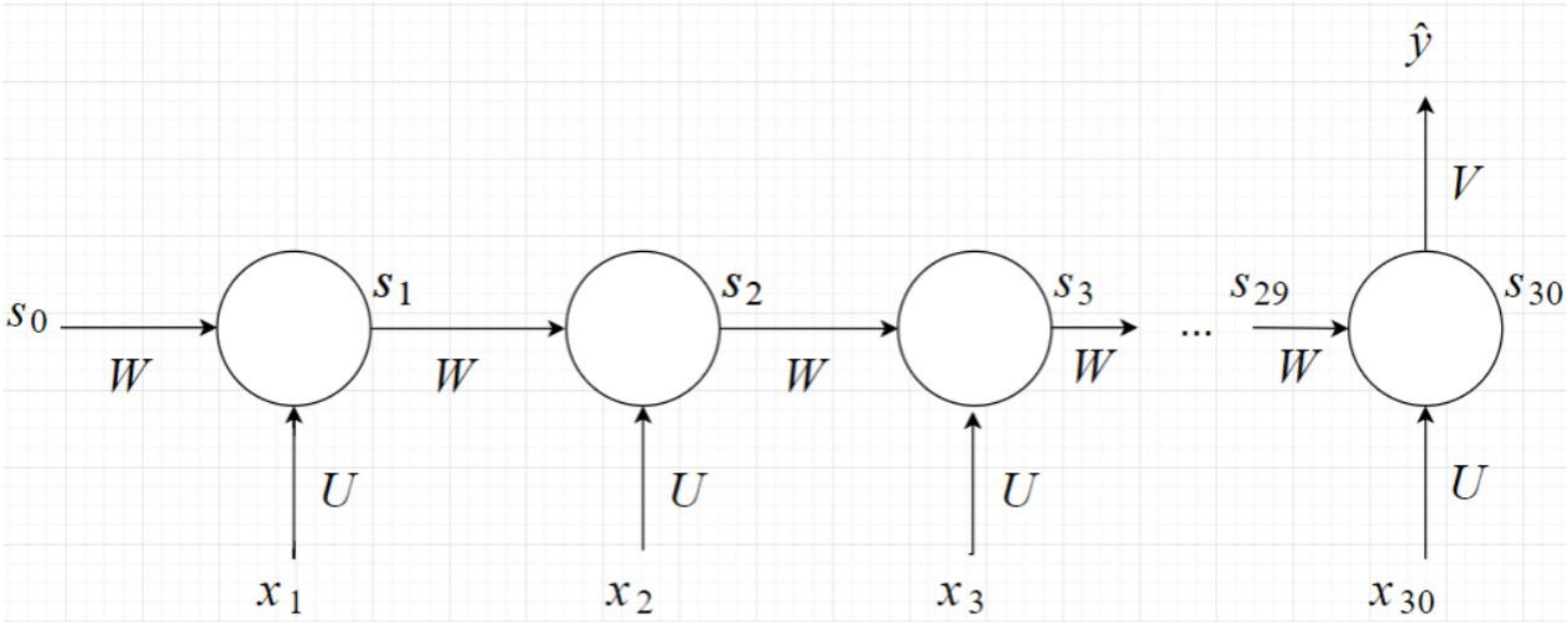
- Ứng dụng của RNN:
  - Về cơ bản, khi thấy sequence data hay time-series data và muốn áp dụng deep learning thì có thể nghĩ ngay đến RNN. Dưới đây là một số ứng dụng của RNN:
    - Speech to text: Chuyển giọng nói sang text.
    - Sentiment classification: Phân loại bình luận của người dùng, tích cực hay tiêu cực.
    - Machine translation: Bài toán dịch tự động giữa các ngôn ngữ.
    - Video recognition: Nhận diện hành động trong video.
    - Heart attack: Dự đoán đột quỵ tim.

Mô hình RNN

- Bài toán: Nhận diện hành động trong video 30s. Đây là dạng bài toán many to one trong RNN, tức nhiều input và 1 output.
  - Tách input video thành 30 ảnh (mỗi giây một ảnh). Các ảnh sẽ được cho qua CNN huấn luyện để lấy ra các feature (feature extraction) vector có kích thước n*1. Vector tương ứng với ảnh ở giây thứ i là x_i.
  - Output là vector có kích thước d*1 (d là số lượng hành động cần phân loại), softmax function được sử dụng như trong bài phân loại ảnh.

Mô hình RNN

- Mô hình RNN cho bài toán



Mô hình RNN

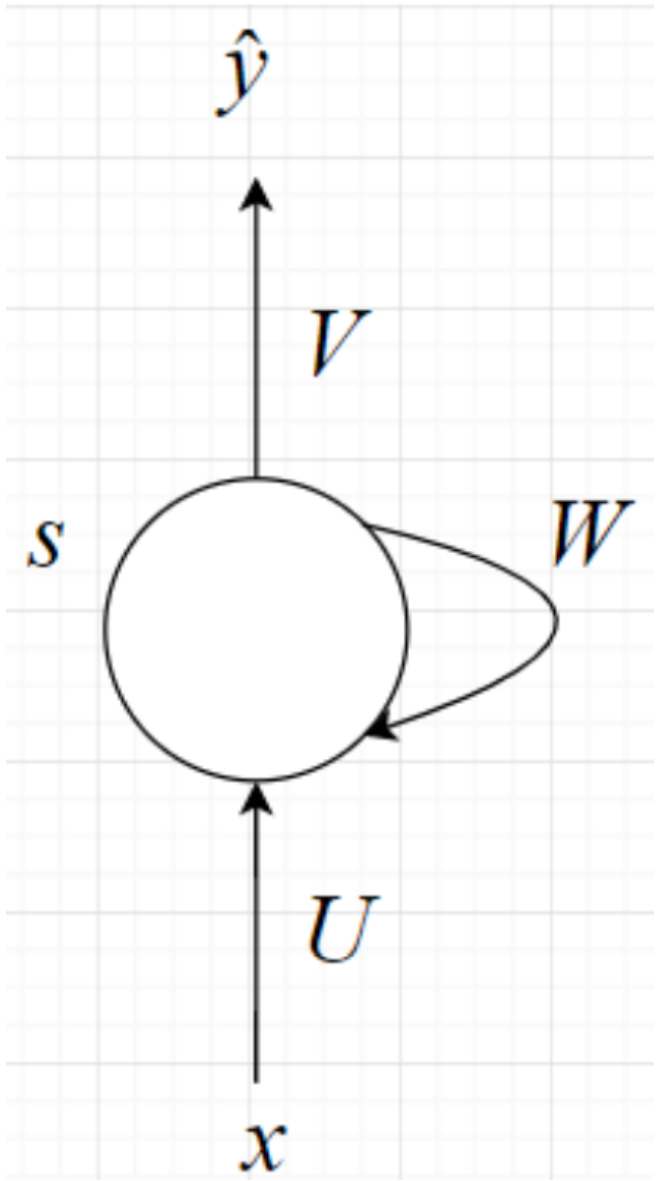
- Mô hình RNN cho bài toán
  - Mô hình có 30 input và 1 output, các input được cho vào model đúng với thứ tự ảnh trong video x_1, x_2, …, x_{30}.
  - Mỗi hình tròn được gọi là 1 state, state t có input là x_t và s_{t-1} (output của state trước), output là $s_t = f(U*x_t + W*s_{t-1})$. f là activation function và thường là Tanh hoặc ReLU.
  - Có thể thấy s_t mang cả thông tin từ state trước (s_{t-1}) và input của state hiện tại => s_t giống như memory nhớ các đặc điểm của các input từ x_1 đến x_t.

Mô hình RNN

- Mô hình RNN cho bài toán
  - Ta thấy là ở mỗi state các hệ số W, U là giống nhau nên model có thể được viết lại thành:
    - x_i là vector có kích thước n*1,
    - s_i là vector có kích thước m*1,
    - y_i là vector có kích thước d*1.
    - U là ma trận có kích thước m*n,
    - W là ma trận có kích thước m*m
    - V là ma trận có kích thước d*m.

$s_0 = 0, s_t = f(U * x_t + W * s_{t-1})$ với $t \geq 0$

$\hat{y} = g(V * s_{30})$



Mô hình RNN

- Loss function
  - Loss function của cả mô hình bằng tổng loss của mỗi output, tuy nhiên ở mô hình trên chỉ có 1 output và là bài toán phân loại nên categorical cross entropy loss sẽ được sử dụng

Mô hình RNN

- Backpropagation Through Time (BPTT)
  - Có 3 tham số ta cần phải tìm là W, U, V. Để thực hiện gradient descent, ta cần tính: $\frac{\partial L}{\partial U}$, $\frac{\partial L}{\partial V}$, $\frac{\partial L}{\partial W}$
  - Ta có:

$$\frac{\partial L}{\partial V} = \frac{\partial L}{\partial \hat{y}} * \frac{\partial \hat{y}}{\partial V}$$

$$\frac{\partial L}{\partial W} = \frac{\partial L}{\partial \hat{y}} * \frac{\partial \hat{y}}{\partial s_{30}} * \frac{\partial s_{30}}{\partial W}$$

Mô hình RNN

- Backpropagation Through Time (BPTT)
  - Có 3 tham số ta cần phải tìm là W, U, V. Để thực hiện gradient descent, ta cần tính: $\frac{\partial L}{\partial U}$, $\frac{\partial L}{\partial V}$, $\frac{\partial L}{\partial W}$
  - Ta có:

$$\frac{\partial L}{\partial V} = \frac{\partial L}{\partial \hat{y}} * \frac{\partial \hat{y}}{\partial V}$$

$$\frac{\partial L}{\partial W} = \frac{\partial L}{\partial \hat{y}} * \frac{\partial \hat{y}}{\partial s_{30}} * \frac{\partial s_{30}}{\partial W}$$

Do $s_{30} = f(W * s_{29} + U * x_{30})$ có s_{29} phụ thuộc vào vào W.

Mô hình RNN

- Backpropagation Through Time (BPTT)
  - Áp dụng công thức $(f(x) * g(x))' = f'(x) * g(x) + f(x) * g'(x)$, ta có:
  - Ta có: $\frac{\partial s_{30}}{\partial W} = \frac{\partial s'_{30}}{\partial W} + \frac{\partial s_{30}}{\partial s_{29}} * \frac{\partial s_{29}}{\partial W}$, trong đó $\frac{\partial s'_{30}}{\partial W}$ là đạo hàm của s_{30} với W khi coi s_{29} là hằng với W.
  - Tương tự, trong biểu thức s_{29} có s_{28} phụ thuộc vào W, s_{28} có s_{27} phụ thuộc vào W, ... nên áp dụng công thức trên và chain rule:

$$\frac{\partial L}{\partial W} = \sum_{i=0}^{30} \frac{\partial L}{\partial \hat{y}} * \frac{\partial \hat{y}}{\partial s_{30}} * \frac{\partial s_{30}}{\partial s_i} * \frac{\partial s'_i}{\partial W}$$, trong đó $\frac{\partial s_{30}}{\partial s_i} = \prod_{j=i}^{29} \frac{\partial s_{j+1}}{\partial s_j}$ và $\frac{\partial s'_i}{\partial W}$ là đạo hàm của s_i với W khi coi s_{i-1} là constant với W.

Mô hình RNN

- Backpropagation Through Time (BPTT)
  - Giả sử activation là tanh function, $s_t = \tanh(U * x_t + W * s_{t-1})$

$$\frac{\partial s_t}{\partial s_{t-1}} = (1 - s_t^2) * W => \frac{\partial s_{30}}{\partial s_i} = W^{30-i} * \prod_{j=i}^{29}(1 - s_j^2).$$

  - Ta có: $s_j < 1, W < 1$ ➔ ở những state xa thì $\frac{\partial s_{30}}{\partial s_i} \approx 0$ hay $\frac{\partial L}{\partial W} \approx 0$. Đây là hiện tượng triệt tiêu gradient (vanishing gradient).
  - Ta có thể thấy, các state càng xa ở trước đó thì càng bị vanishing gradient và các hệ số không được update với các frame ở xa. Hay nói cách khác là RNN không học được từ các thông tin ở trước đó xa do vanishing gradient.

    => Long short term memory (LSTM) ra đời.

Long short term memory (LSTM)

- Về lý thuyết, RNN có thể mang thông tin từ các layer trước đến các layer sau, nhưng thực tế là thông tin chỉ mang được qua một số lượng state nhất định, sau đó thì sẽ bị vanishing gradient, hay nói cách khác là model chỉ học được từ các state gần nó => short term memory.

Long short term memory (LSTM)

- Bài toán là dự đoán từ tiếp theo trong đoạn văn.
  - Đoạn, "Mặt trời mọc ở hướng ...", ta có thể chỉ sử dụng các từ trước trong câu để đoán là đông.
  - Với đoạn, "Tôi là người Việt Nam. Tôi đang sống ở nước ngoài. Tôi có thể nói trôi chảy tiếng ... " thì rõ ràng là chỉ sử dụng từ trong câu đấy hoặc câu trước là không thể dự đoán được từ cần điền là Việt.

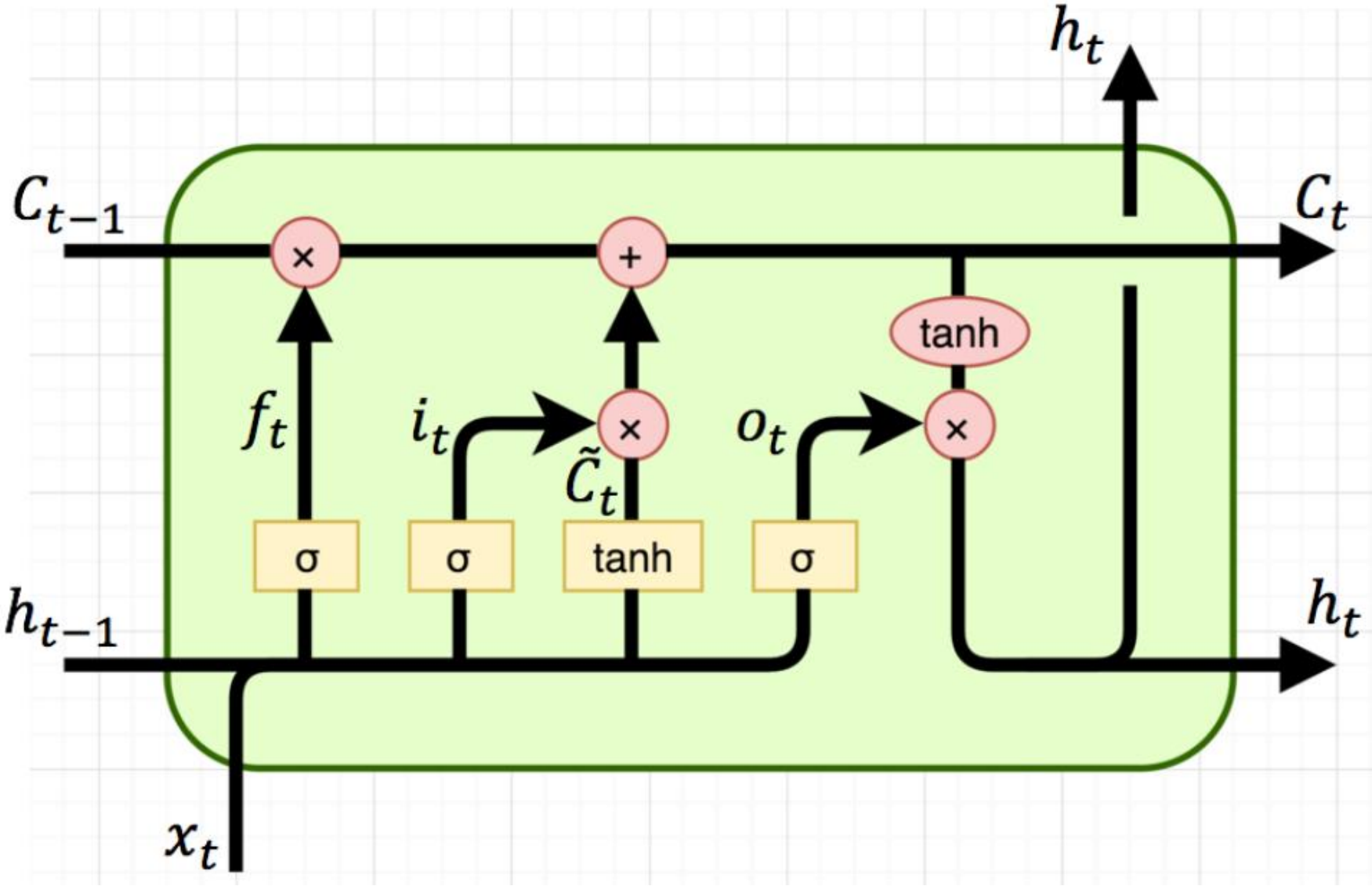
    Ta cần các thông tin từ state ở trước đó rất xa => cần long term memory mà RNN không làm được => Cần một mô hình mới để giải quyết vấn đề này
    => Long short term memory (LSTM) ra đời.

Mô hình LSTM

- Ở state thứ t của mô hình LSTM:
  - Output: c_t, h_t, ta gọi c là cell state, h là hidden state.
  - Input: c_{t-1}, h_{t-1}, x_t. Trong đó, x_t là input ở state thứ t của model. c_{t-1}, h_{t-1} là output của layer trước. h đóng vai trò khá giống như s ở RNN, trong khi c là điểm mới của LSTM.

    ➔ Sử dụng thêm "cell" có bộ nhớ để tránh hiện tượng triệt tiêu gradient.

Mô hình LSTM

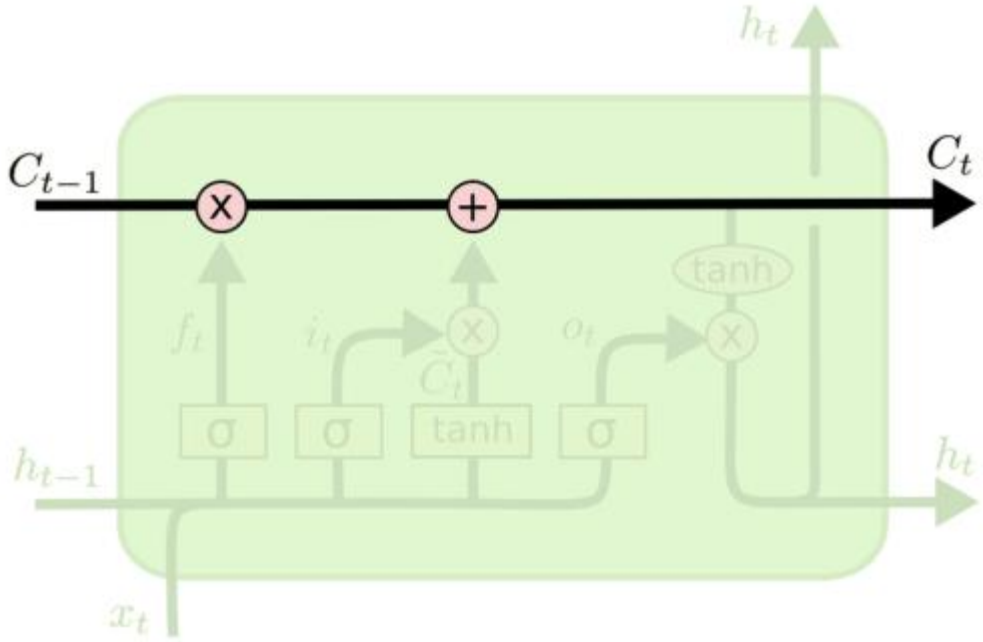


Mô hình LSTM

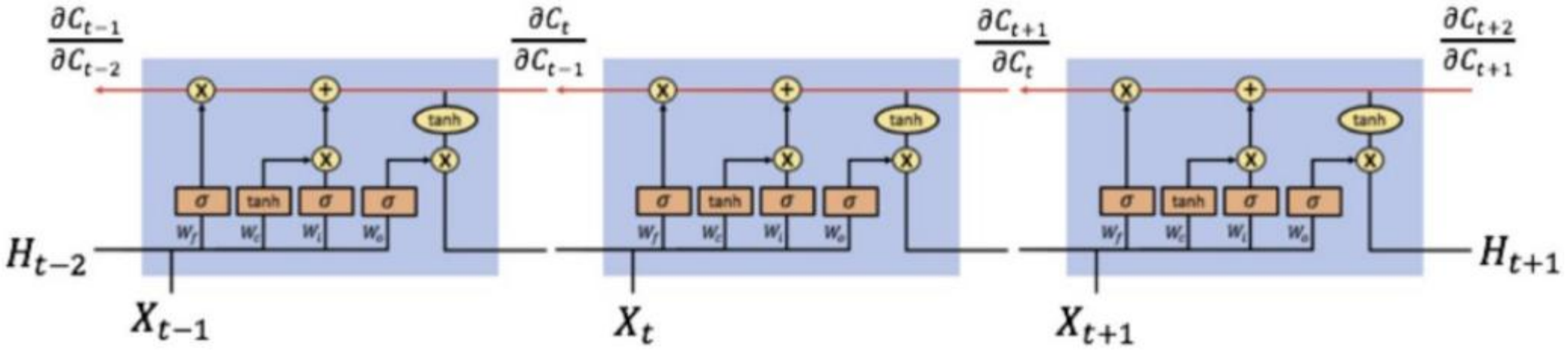
- Kí hiệu σ, tanh ý là bước đấy dùng sigma, tanh activation function.
- Phép nhân ở đây là element-wise multiplication, phép cộng là cộng ma trận.
- f_t, i_t, o_t tương ứng với forget gate, input gate và output gate.
  - Forget gate: $f_t = \sigma(U_f * x_t + W_f * h_{t-1} + b_f)$
  - Input gate: $i_t = \sigma(U_i * x_t + W_i * h_{t-1} + b_i)$
  - Output gate: $o_t = \sigma(U_o * x_t + W_o * h_{t-1} + b_o)$
- Nhận xét: $0 < f_t, i_t, o_t < 1$; b_f, b_i, b_o là các hệ số bias; hệ số W, U giống như RNN
- $\tilde{c}_t = \tanh(U_c * x_t + W_c * h_{t-1} + b_c)$, giống như tính s_t trong CNN
- $c_t = f_t * c_{t-1} + i_t * \tilde{c}_t$, **forget gate** quyết định xem cần lấy bao nhiêu từ cell state trước và **input gate** sẽ quyết định lấy bao nhiêu từ input của state và hidden layer của layer trước

Mô hình LSTM

- $h_t = o_t * \tanh(c_t)$, **output gate** quyết định xem cần lấy bao nhiêu từ cell state để trở thành output của hidden state. Ngoài ra, h_t cũng được dùng để tính ra output y_t cho state t.
- Nhận xét:
  - $h_t, \tilde{c}_t$ khá giống với RNN nên model có short term memory.
  - c_t giống như một băng chuyền ở trên mô hình RNN, thông tin nào cần quan trọng và dùng ở sau sẽ được gửi vào và dùng khi cần => có thể mang thông tin từ đi xa => long term memory. Do đó, mô hình LSTM có cả short term memory và long term memory.



LSTM chống vanishing gradient



Thành phần chính gây là vanishing gradient trong RNN là $\frac{\partial s_{t+1}}{\partial s_t} = (1 - s_t^2) * W$, trong đó $s_t, W < 1$.

Tương tự trong LSTM ta quan tâm đến $\frac{\partial c_t}{\partial c_{t-1}} = f_t$. Do $0 < f_t < 1$ nên về cơ bản thì LSTM vẫn bị vanishing gradient nhưng bị ít hơn so với RNN. Hơn thế nữa, khi mang thông tin trên cell state thì ít khi cần phải quên giá trị cell cũ, nên $f_t \approx 1$ => Tránh được vanishing gradient.

Gated Recurrent Unit (GRU)

- Không dùng “cell state” riêng biệt, ghép chung với hidden state
- Kết hợp cổng “forget” và “output” thành cổng “update”

$$z_t = \sigma(W_z \cdot [h_{t-1}, x_t])$$

σ là hàm sigmoid

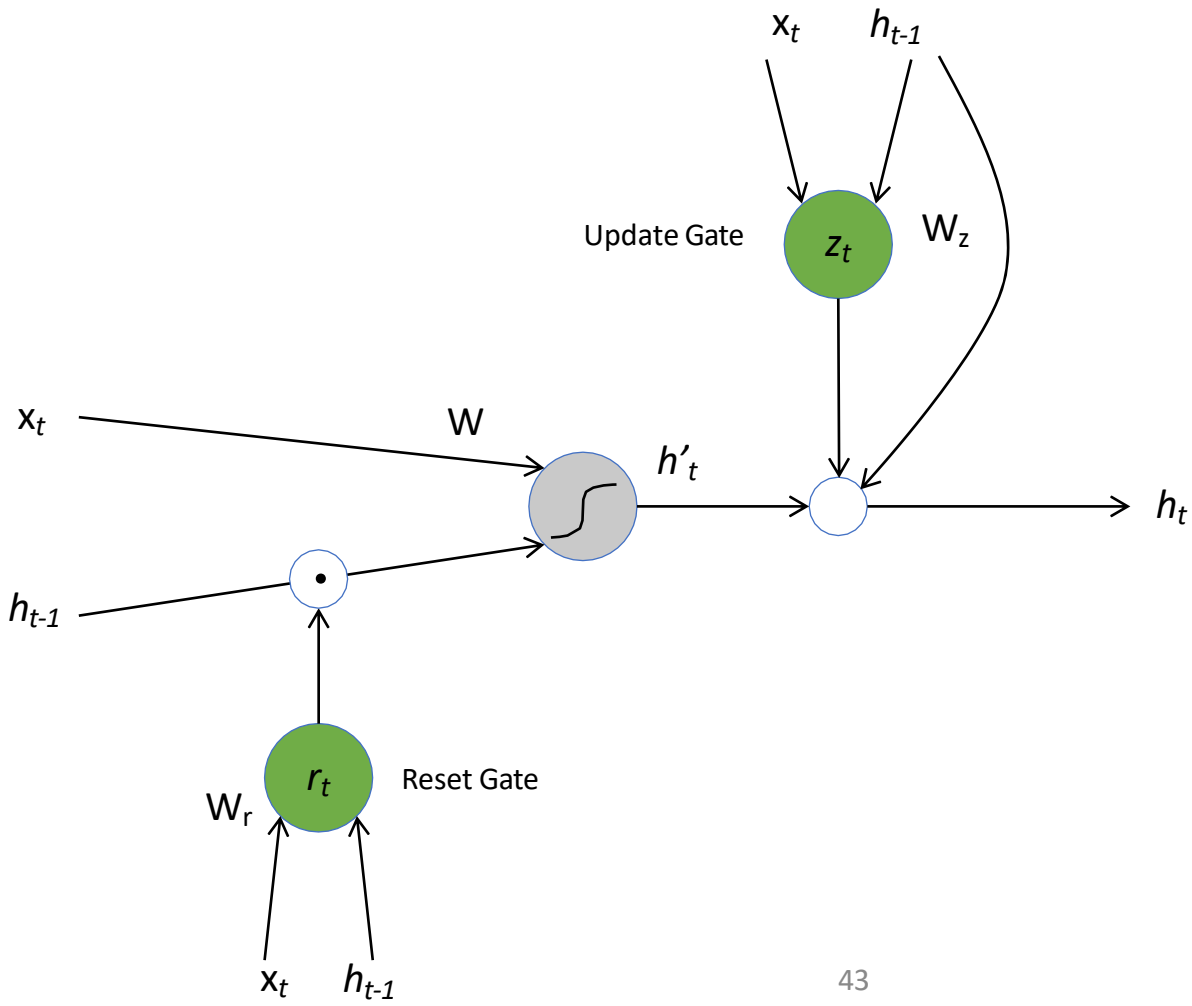
W_z là các trọng số của update gate

Đầu là là số có giá trị từ 0 đến 1

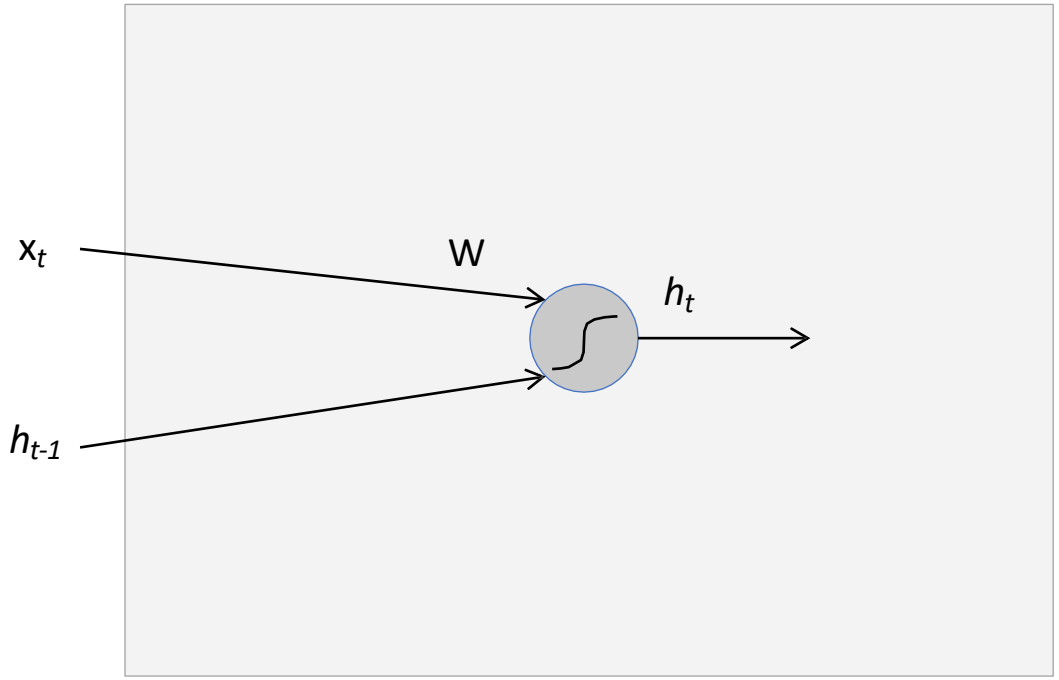
$$r_t = \sigma(W_r \cdot [h_{t-1}, x_t])$$

Cổng Reset Gate xác định mức độ trạng thái ẩn trước đó sẽ bị quên.

Khi r_t gần 0 thì mạng quên quá khứ

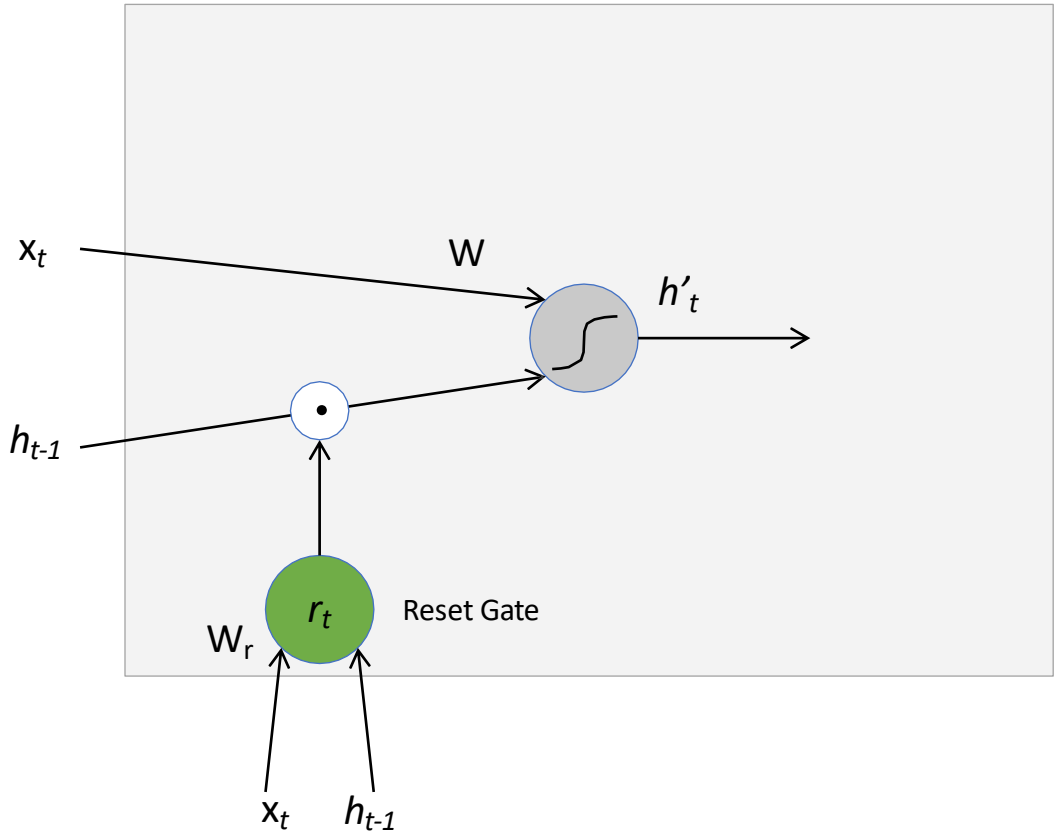


Gated Recurrent Unit (GRU)



$$h_t = \tanh W \begin{pmatrix} x_t \\ h_{t-1} \end{pmatrix}$$

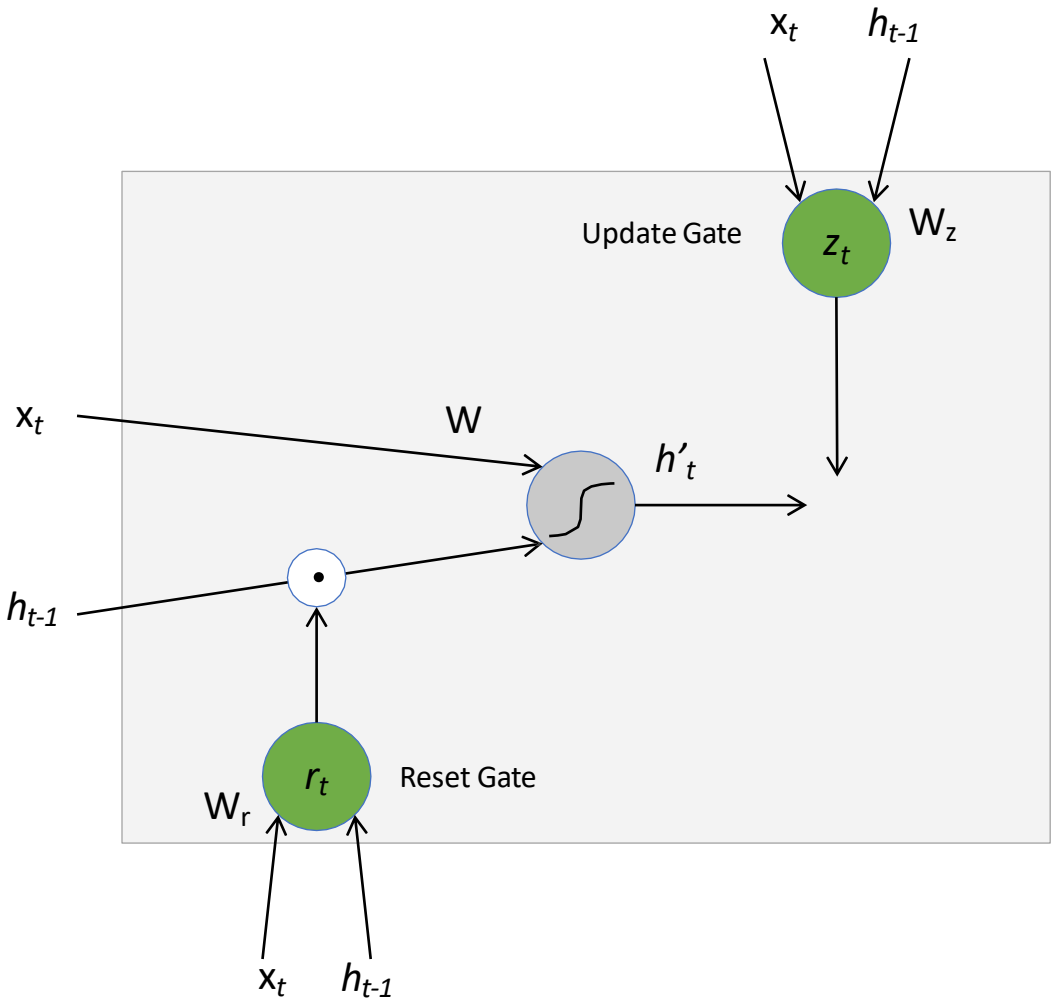
Gated Recurrent Unit (GRU)



$$r_t = \sigma\left(W_r\begin{pmatrix} x_t \\ h_{t-1} \end{pmatrix} + b_t\right)$$

$$h_t' = \tanh W\begin{pmatrix} x_t \\ r_t \odot h_{t-1} \end{pmatrix}$$

Gated Recurrent Unit (GRU)

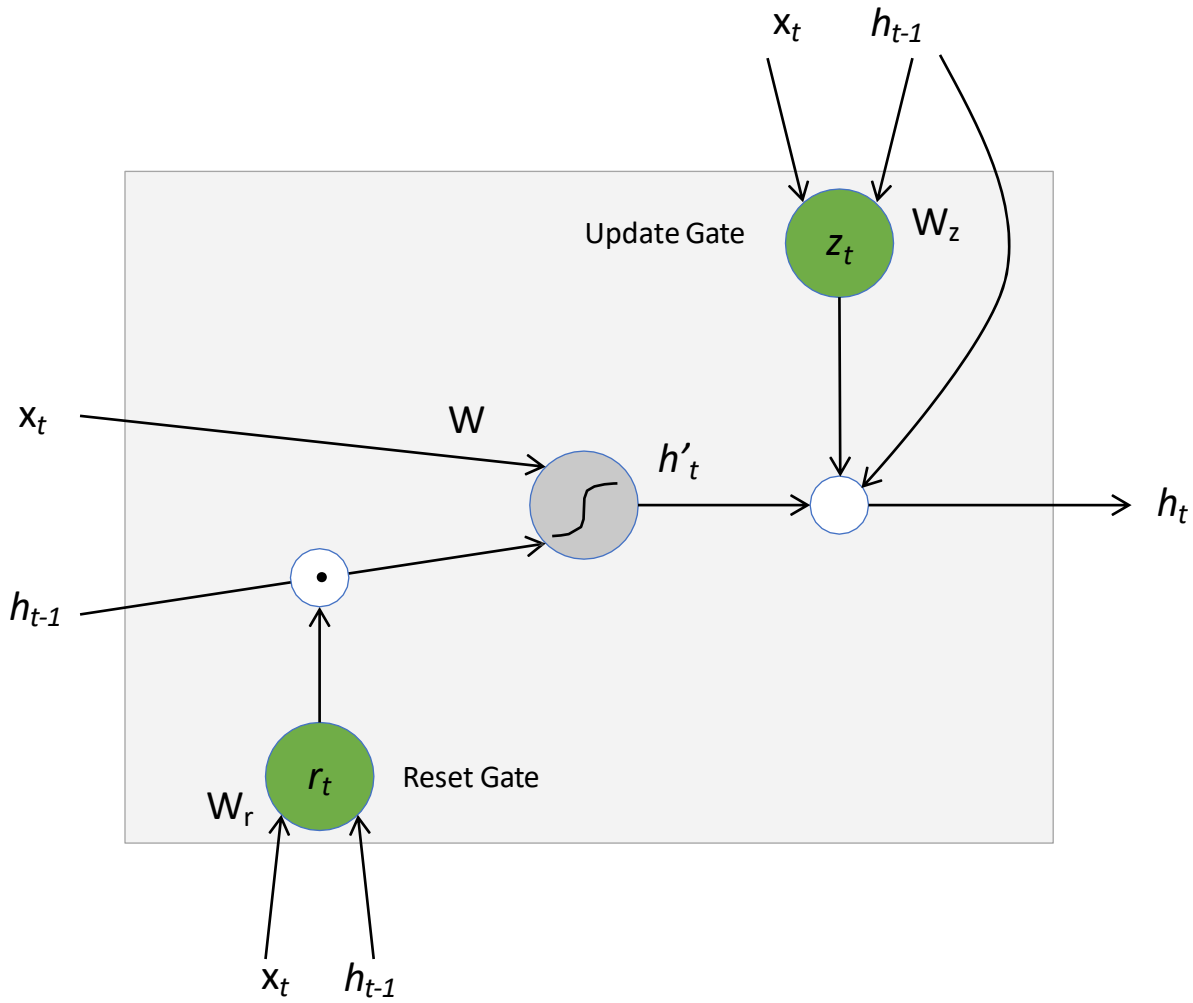


$$r_t = \sigma\left(W_r\begin{pmatrix} x_t \\ h_{t-1} \end{pmatrix} + b_t\right)$$

$$h'_t = \tanh W\begin{pmatrix} x_t \\ r_t \odot h_{t-1} \end{pmatrix}$$

$$z_t = \sigma\left(W_z\begin{pmatrix} x_t \\ h_{t-1} \end{pmatrix} + b_z\right)$$

Gated Recurrent Unit (GRU)



$$r_t = \sigma\left(W_r\begin{pmatrix}x_t\\h_{t-1}\end{pmatrix} + b_t\right)$$

$$h'_t = \tanh W\begin{pmatrix}x_t\\r_t \odot h_{t-1}\end{pmatrix}$$

$$z_t = \sigma\left(W_z\begin{pmatrix}x_t\\h_{t-1}\end{pmatrix} + b_z\right)$$

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot h'_t$$